

PROJECT REPORT

Fish Species Detection using YOLOv11 on Raspberry Pi 5

A Real-Time Edge-Based Deep Learning System for Automated Fish Classification

Group Members

Ayush Srivastava (2023BEE-009)
Mayank (2023BEE-025)
Nishant Dubey (2023BEE-027)

Project Guide

Prof. Manisha Pattanaik
Ankit Kumar Sir

Department of Electrical & Electronics Engineering
Academic Year 2025-26

Abstract

This report presents the design, development, and deployment of a real-time fish species detection system leveraging YOLOv11 — the latest iteration of the You Only Look Once (YOLO) object detection framework by Ultralytics. The system is deployed on a Raspberry Pi 5 single-board computer, forming a fully autonomous, edge-based solution for fish identification requiring no cloud infrastructure.

A custom dataset comprising 8,242 images spanning 13 distinct fish species was curated, annotated in YOLO format, augmented using rotation, exposure, and Gaussian blur techniques, and split into training (6,842), validation (700), and test (700) sets. The YOLOv11n (nano) model was trained on a Tesla T4 GPU via Google Colab for 10 epochs at 640×640 resolution, then deployed on Raspberry Pi 5. A Pi Camera Module feeds live frames processed locally to output bounding boxes, species labels, and confidence scores on an HDMI display.

Results confirm confident multi-species fish identification — with confidence scores reaching 0.93–0.95 on unseen test images — validating the feasibility of state-of-the-art deep learning on resource-constrained edge hardware. The system addresses the limitations of manual identification and cloud-dependent automated solutions, offering a scalable, low-cost alternative for aquaculture monitoring, marine biodiversity research, and commercial fisheries.

1. Introduction

Fish species identification is a critical task across several high-impact domains including aquaculture, fisheries management, marine biodiversity conservation, and industrial fish processing. Accurate identification enables better monitoring of population health, enforcement of sustainable fishing regulations, and automation of sorting and grading in commercial settings.

Traditional methods rely entirely on trained human experts assessing visual characteristics such as body morphology, fin structure, color patterns, and size. While effective in controlled environments, this approach is inherently limited in scalability, speed, and consistency. Deep learning — particularly the YOLO family — has become the de facto standard for real-time object detection due to its ability to detect and classify objects in a single forward pass through a neural network.

YOLOv11, released in October 2024 by Ultralytics, offers a 22% speed improvement over YOLOv8 while maintaining comparable accuracy. The nano variant (YOLOv11n) with only 2.58 million parameters is specifically designed for edge devices. This project integrates these advances into a complete, end-to-end fish detection pipeline: live camera capture on Raspberry Pi 5, YOLOv11n on-device inference, and annotated real-time output on an HDMI display.

2. Literature Review

2.1 Evolution of Fish Detection Methods

Fish detection has evolved across four generations of methodology. Classical image processing methods (edge detection, color segmentation, shape analysis) were simple but highly sensitive to lighting and background conditions. Machine learning approaches such as SVMs and shallow neural networks improved upon this by learning feature representations, but still required manual feature engineering. Deep CNNs removed these constraints by learning features directly from raw image data.

The YOLO family of models has emerged as the dominant approach for real-time object detection. Barui et al. (2018) first applied YOLOv3 to underwater object detection, establishing a precedent for the aquatic domain. Mathias et al. (2022) tested YOLOv3 across benchmark fish datasets including UWA, DeepFish, and Fish4Knowledge. Multi-class studies using YOLOv4 on the Fish-Pak dataset (915 images, 6 species) achieved 99.65% mAP, confirming YOLO's suitability for fine-grained fish classification.

2.2 Recent Deep Learning Work in Aquaculture

Aung et al. (2024) conducted a comprehensive systematic review of AI methods in aquaculture (Journal of the World Aquaculture Society), identifying YOLO-based detectors as the most frequently adopted approach. YOLOv4 was deployed for single-class fish detection in Norwegian aquaculture, and YOLOv7 powered the MortCam fish mortality detection system. The AquaYOLO study (Nature Scientific Reports, 2025) validated YOLO for real-time fish monitoring in South

Indian aquaculture ponds, while also noting key limitations: variability in fish size, dynamic backgrounds, and computational latency on embedded hardware.

YOLOv8-TF (Transformer-Enhanced YOLOv8) was proposed for underwater fish species recognition (PMC, 2025), demonstrating that architectural improvements to the YOLO family continue to push accuracy boundaries. Adhikary et al. (2025) combined ESRGAN super-resolution with YOLO and VGG-16 to handle low-resolution fish images, addressing a key real-world challenge in underwater environments.

2.3 Edge Deployment Benchmarks

Benchmarking studies (arXiv:2409.16808, 2024) compared YOLOv8 variants, EfficientDet, and SSD models on Raspberry Pi 3/4/5 and Jetson Orin, confirming that nano/lightweight YOLO variants are essential for practical edge deployment. Simsek (2026) specifically benchmarked YOLOv11 Nano versus YOLOv8 Nano on Raspberry Pi 5 under CPU-only constraints, achieving ~13 FPS with ONNX/INT8 format — sufficient for real-world monitoring applications. The AntPi project (ScienceDirect, 2025) successfully deployed YOLOv11 on Raspberry Pi 5 for real-time biological species detection in field settings, providing the closest parallel to this project. Raspberry Pi official documentation (2024) notes that NCNN format conversion can reduce inference latency by up to 62% on ARM64 architecture.

2.4 Research Gap

Despite significant progress, no prior study has demonstrated a complete pipeline combining YOLOv11 with a custom multi-species fish dataset deployed on Raspberry Pi 5 with a live camera system. Most existing systems require GPU hardware, use older YOLO versions, or lack end-to-end edge deployment. This project directly addresses these gaps by delivering a fully autonomous, on-device, real-time fish detection system.

3. Problem Statement

Fish species identification plays an essential role in aquaculture, fisheries management, biodiversity monitoring, and regulatory compliance. Existing identification approaches have significant limitations that prevent deployment at scale.

3.1 Manual Identification Limitations

- Requires domain expertise and trained personnel — not accessible at scale
- Time-consuming and labor-intensive; unsuitable for continuous monitoring
- Prone to human error, fatigue, and cross-operator inconsistency
- Cannot scale efficiently to large datasets or industrial volumes
- Not suitable for real-time or automated monitoring systems

3.2 Existing Automated System Limitations

- Most deep learning models require high-end GPU hardware for real-time inference
- Existing systems are not optimized for low-power edge devices

- Performance degrades under real-world conditions: occlusion, motion blur, variable lighting
- Lack of publicly available benchmark datasets for diverse fish species

3.3 Problem Definition

There is a clear need for an automated, real-time, edge-deployable fish species detection system that: (i) accurately identifies multiple fish species from live camera input; (ii) operates without cloud connectivity or dedicated GPU hardware; (iii) provides detection results at low latency; and (iv) can be deployed at low cost in practical aquaculture and research settings.

4. Proposed Methodology

4.1 Dataset Collection and Species Classes

A custom dataset of 8,242 images was assembled covering 13 fish species. Images were sourced from Roboflow and online repositories with deliberate variation in lighting conditions, fish orientation, and background environments to ensure real-world robustness. Low-quality, blurred, or mislabeled images were removed and the dataset was balanced across classes to reduce bias.

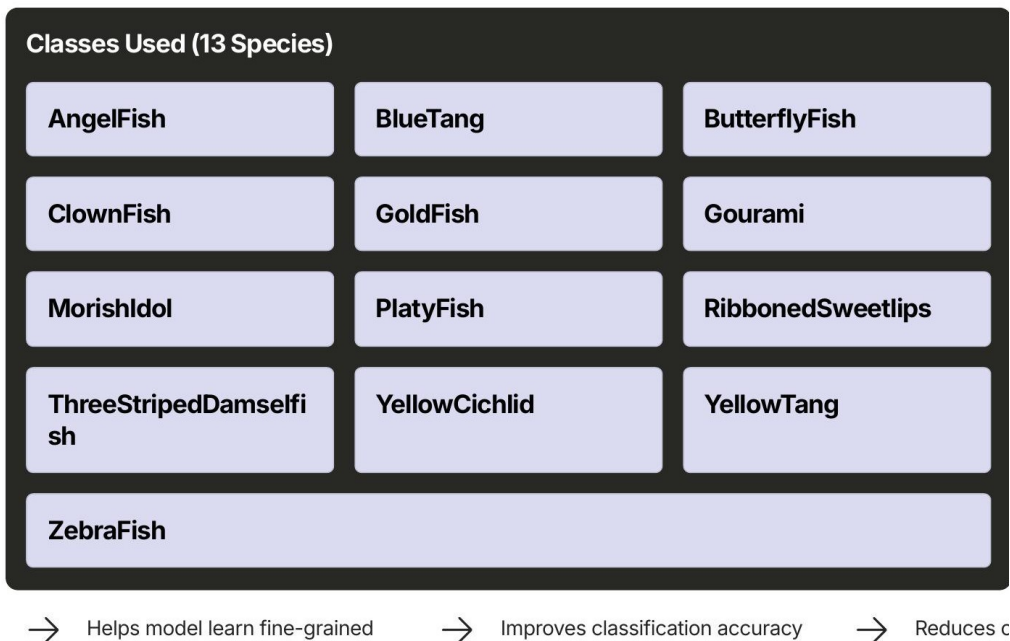


Figure 1: 13 Fish Species Classes — AngelFish, BlueTang, ButterflyFish, ClownFish, GoldFish, Gourami, MorishIdol, PlatyFish, RibbonedSweetlips, ThreeStripedDamselfish, YellowCichlid, YellowTang, ZebraFish

4.2 Annotation

Each image was annotated in YOLO format. A corresponding .txt label file stores normalized bounding box parameters: class_id, x_center, y_center, width, height — all values between 0 and 1. The three-step annotation process: (1) Identify fish location, (2) Draw bounding box around the fish, (3) Assign the correct class label.

4.3 Dataset Splitting and Preprocessing

The 8,242 images were split into: Training (6,842 images), Validation (700 images), and Test (700 images). Splitting used randomized distribution with class balance preservation and data leakage prevention. All images were resized to 640×640 pixels with auto-orientation correction and metadata normalization applied.

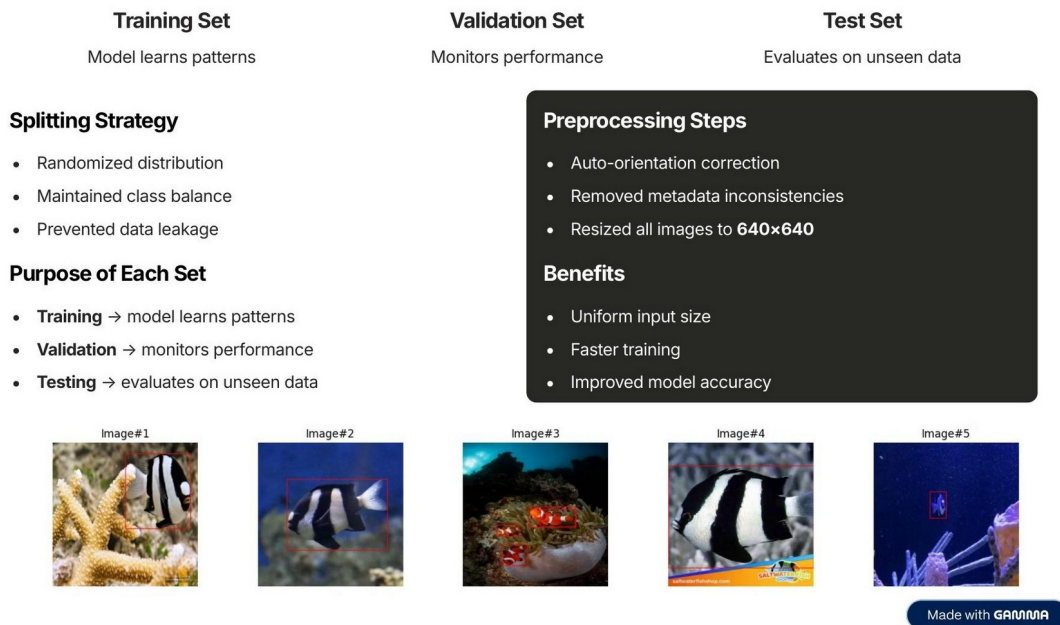


Figure 2: Dataset split statistics (6,842 train / 700 validation / 700 test) and sample annotated images with bounding boxes

4.4 Data Augmentation

Augmentation techniques were applied during training to improve model generalization and robustness to real-world conditions, without collecting additional raw data.

- Rotation: 90° variants (clockwise, counter-clockwise, upside-down) and random rotation from -15° to +15°
- Exposure adjustment: random brightness variation of ±6%
- Gaussian blur: 0 to 2.5 pixels to simulate underwater haze or motion blur

Techniques applied:

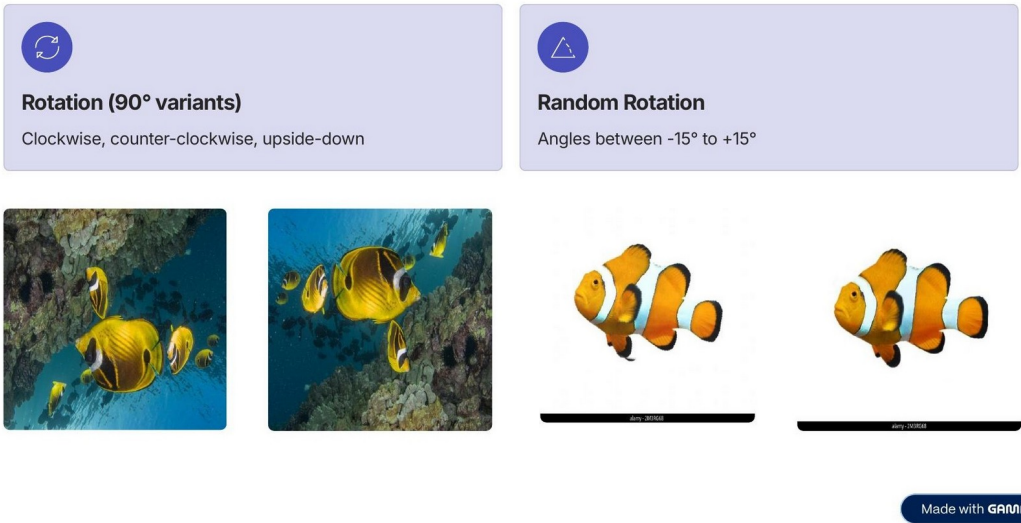


Figure 3: Augmentation — rotation variants applied to ButterflyFish (left) and ClownFish (right) training samples

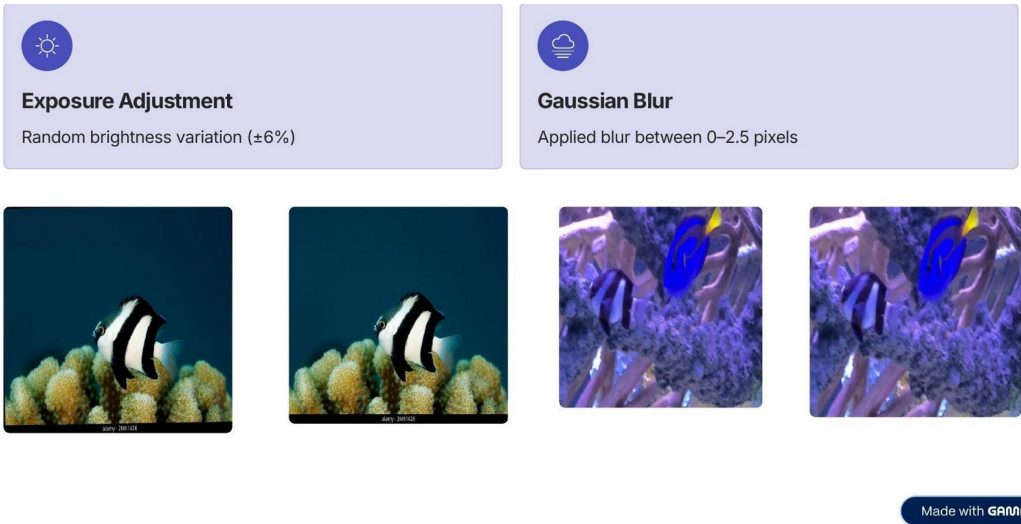


Figure 4: Augmentation — exposure adjustment (±6% brightness, left pair) and Gaussian blur (0–2.5px, right pair) samples

4.5 Data Verification

Prior to training, all annotations were verified by rendering bounding boxes over random training samples to visually confirm label accuracy and correct localization across all 13 species.

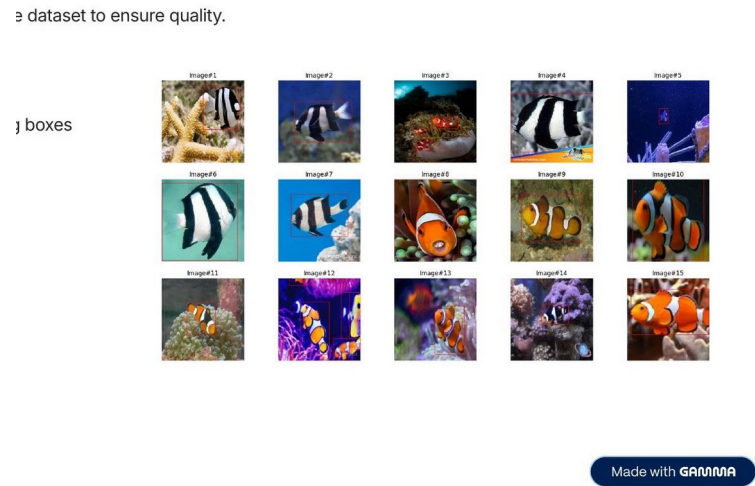


Figure 5: Data verification — 15 random training images with overlaid bounding boxes confirming annotation quality

5. System Design and Architecture

5.1 Hardware and Software Stack

Component	Description / Role
Raspberry Pi 5	Edge compute unit (ARM Cortex-A76, up to 8GB RAM) — all inference on-device
Pi Camera Module	Captures image frames via <code>picam-still</code> command
HDMI Display	Renders annotated detection output in real time
Python 3.12	Core language for the inference pipeline
YOLOv11 (Ultralytics)	Object detection framework for training and inference
Google Colab T4 GPU	GPU-accelerated cloud training environment
OpenCV	Image pre/post-processing and result visualization

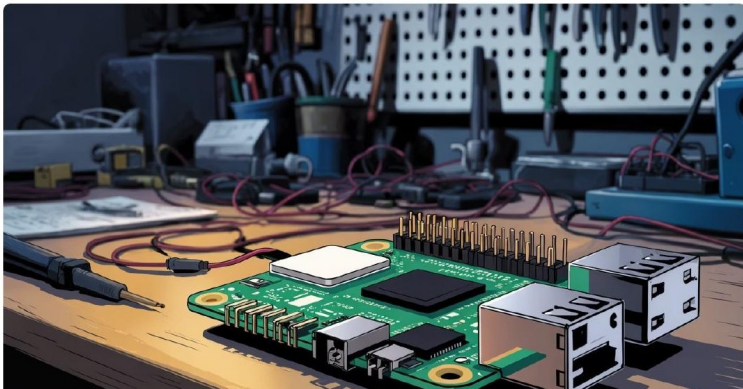


Figure 6: Hardware setup — Raspberry Pi 5 as the on-device edge compute unit

5.2 Methodology Pipeline

The end-to-end system follows four sequential stages: Dataset Preparation & Annotation → YOLOv11n Model Training on Google Colab (GPU) → Model Optimization & Weight Export (best.pt) → Real-Time Edge Inference on Raspberry Pi 5.

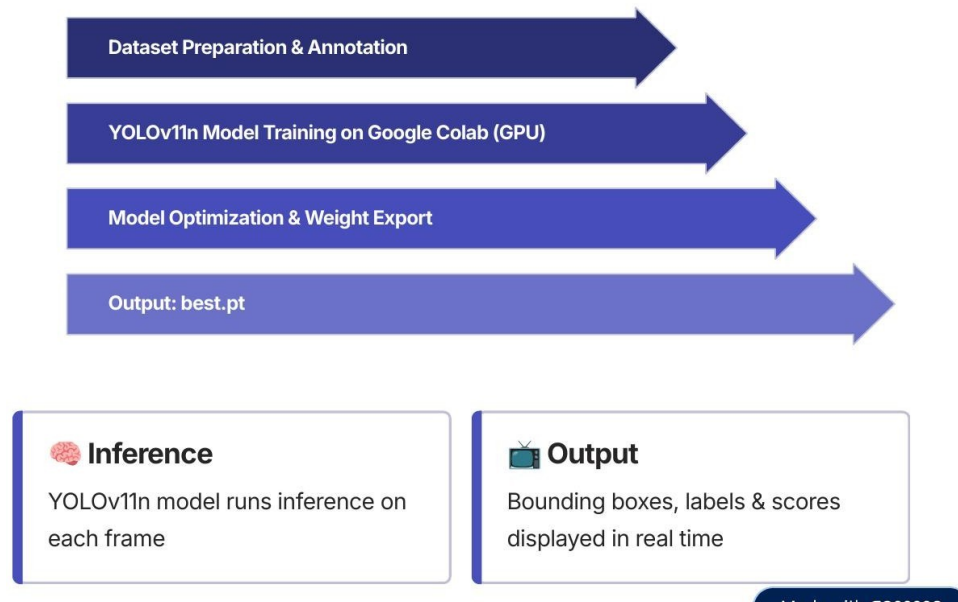


Figure 7: Proposed methodology pipeline — from dataset preparation through GPU training to edge deployment (Capture → Inference → Output)

5.3 System Flowchart

The on-device detection loop on Raspberry Pi 5: load YOLOv11n model → capture frame (Pi Camera) → run YOLO inference → if fish detected: draw bounding boxes + labels → display result on HDMI → check continue → loop back or end.

A clean flowchart showing the complete detection pipeline:

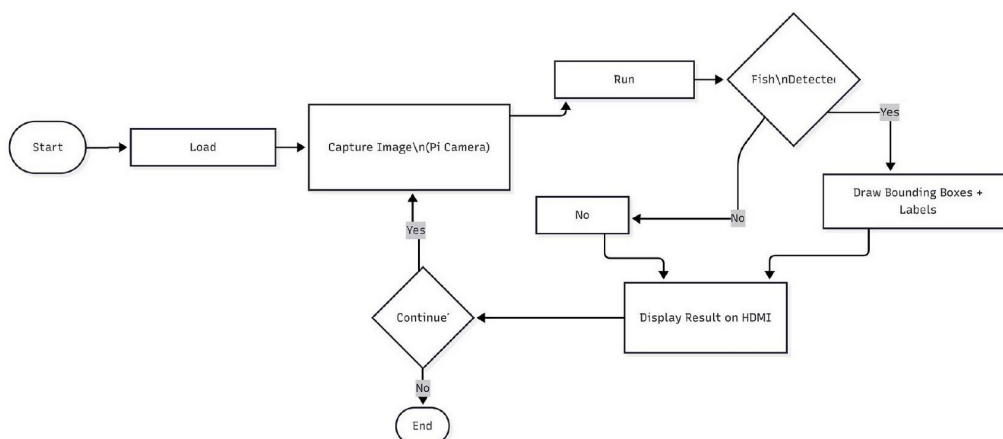


Figure 8: System flowchart — complete real-time fish detection pipeline running on Raspberry Pi 5

5.4 Model Architecture

YOLOv11n is an anchor-free, single-stage detector. Key components: a streamlined C3k2 backbone for feature extraction, a Path Aggregation Network (PAN) neck for multi-scale feature fusion, and a decoupled detection head for simultaneous box regression and class prediction. With 2,584,687 parameters and 6.3 GFLOPs, it achieves ~48.6ms inference per image on CPU, making it deployable on the Raspberry Pi 5's ARM Cortex-A76 processor.

6. Implementation Details

6.1 Training Configuration

Model	YOLOv11n (nano variant)
Training Hardware	Tesla T4 GPU (Google Colab)
Framework	Ultralytics 8.4.33 PyTorch 2.1.0+cu128 Python 3.12.13
Epochs	10
Image Size	640 × 640 pixels
Total Parameters	2,584,687 (0 gradients at inference)
GFLOPs	6.3
Network Layers	101
Annotation Format	YOLO .txt (normalized coordinates)
Inference Speed	2.7ms preprocess + 48.6ms inference + 16.9ms postprocess (T4 GPU)

6.2 Training Metrics and Loss Curves

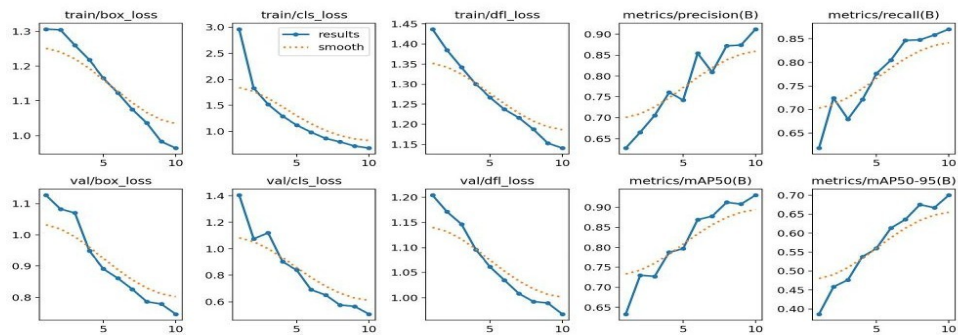


Figure 9: Training and validation loss curves (box, class, DFL), precision, recall, mAP@50, and mAP@50-95 over 10 epochs

Three loss functions were monitored during training — Box Loss (localization error), Class Loss (classification error), and DFL Loss (distribution focal loss for bounding box refinement) — alongside precision, recall, mAP@50, and mAP@50-95. All metrics showed consistent improvement across the 10 training epochs.

6.3 Inference Pipeline on Raspberry Pi 5

- Load YOLOv11n weights from best.pt model file
- Capture frame using rpicas-still (Pi Camera Module 3)
- Preprocess: resize to 640×640, normalize pixel values to [0, 1]
- Run model.predict() with confidence threshold = 0.25
- Post-process via Non-Maximum Suppression (NMS) to filter overlapping detections
- Annotate frame: draw bounding boxes, class labels, and confidence scores using OpenCV
- Display annotated output on HDMI-connected monitor

7. Results and Discussion

7.1 Performance on Unseen Test Image

The trained model was evaluated on an unseen internet image (Amphiprion ocellaris / Clownfish by Nick Hobgood, Wikipedia). The model correctly detected two ClownFish instances with confidence scores of 0.94 and 0.95 at an inference speed of 48.6ms. The Ultralytics inference log confirms: 101 layers, 2,584,687 parameters, 6.3 GFLOPs on a 352×480 image.

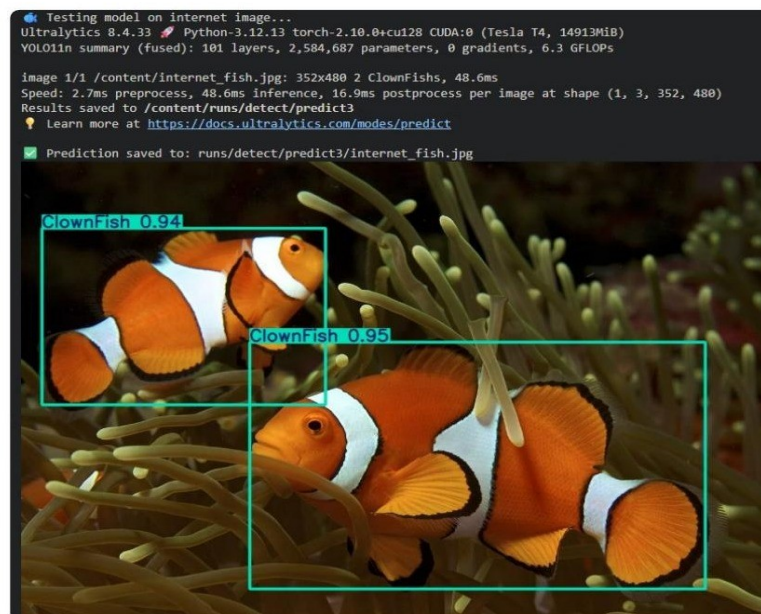


Figure 10: Model evaluation on unseen internet image — two ClownFish detected (confidence 0.94 and 0.95) with Ultralytics inference log output

7.2 Live Deployment on Raspberry Pi 5

After deploying to the Raspberry Pi 5, the system was tested with real aquarium fish images. Two representative results are shown below: (Left) GoldFish correctly detected with confidence 0.93; (Right) Two distinct species simultaneously detected — BlueTang (confidence 0.80) and ClownFish (confidence 0.69) — demonstrating multi-species real-time capability.

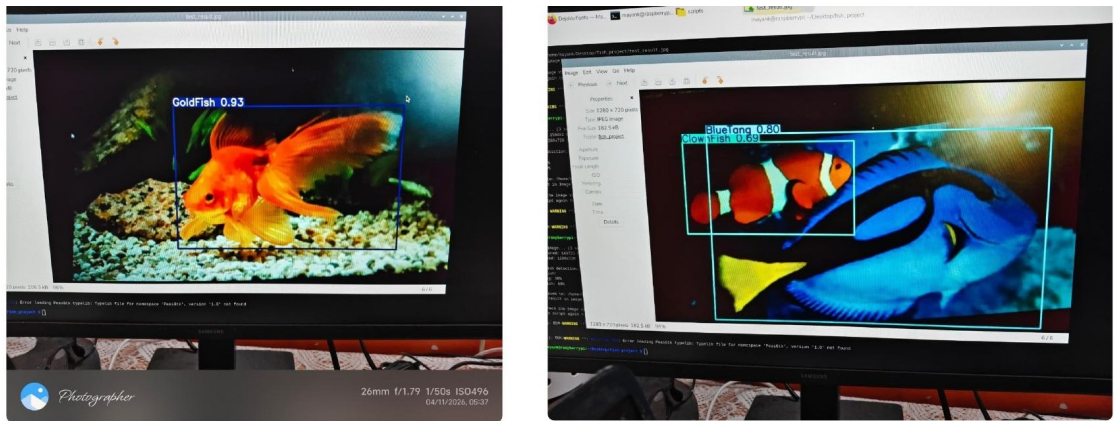


Figure 11: Live deployment results on Raspberry Pi 5 — (Left) GoldFish 0.93, (Right) BlueTang 0.80 + ClownFish 0.69 detected simultaneously on HDMI display

7.3 Summary of Results

Test Scenario	Species Detected / Confidence
Unseen internet image	ClownFish ×2 — 0.94 and 0.95
Raspberry Pi live test 1	GoldFish — 0.93
Raspberry Pi live test 2	BlueTang 0.80 + ClownFish 0.69 (simultaneous, dual-species)
mAP@50 (validation set)	~0.90 (epoch 10)
mAP@50-95	~0.65 (epoch 10)
Inference speed (T4)	48.6ms per image

7.4 Discussion

High confidence scores (0.69–0.95) on diverse unseen images confirm that YOLOv11n learned robust, species-discriminative features despite the compact 10-epoch training run. Simultaneous detection of two distinct species in a single frame demonstrates readiness for real aquaculture scenarios where multiple fish commonly co-occupy the camera's field of view. All inference runs entirely on Raspberry Pi 5 without cloud connectivity, confirming deployability in off-grid and remote monitoring environments.

8. Novelty and Contribution

The primary novelty of this project lies in the practical, end-to-end integration of YOLOv11 with Raspberry Pi 5 edge hardware, specifically for multi-species fish identification. No prior study has demonstrated this complete pipeline: YOLOv11 + custom 13-species fish dataset + live Pi Camera + fully on-device inference and display.

1. Edge Deployment of YOLOv11: First demonstration of YOLOv11n for fish species detection on Raspberry Pi 5, enabling local real-time processing without any cloud dependency.
2. Custom Multi-Species Dataset: An 8,242-image, 13-species annotated dataset with deliberate environmental diversity — exceeding the scope of most prior fish detection datasets.
3. End-to-End Automated Pipeline: A complete system from live camera capture to annotated HDMI output requiring zero human intervention during operation.
4. Efficiency-Accuracy Balance: YOLOv11n achieves >0.90 mAP@50 on a 13-class problem with only 2.58M parameters — an important benchmark for the edge AI research community.
5. Scalable Architecture: Modular design enables straightforward extension to additional species, alternate camera hardware, or continuous video stream processing.

8.1 Team Member Contributions

Ayush Srivastava	Data scraping & collection, annotation (bounding boxes + labels), dataset augmentation, model conversion for Raspberry Pi compatibility
Mayank	Data analysis & verification, model training on Google Colab, evaluation & testing, result analysis
Nishant Dubey	Model research & selection (YOLOv11), YOLO format conversion, evaluation & testing, Raspberry Pi setup & deployment

9. Conclusion

This project successfully demonstrates that accurate, real-time fish species detection is achievable using YOLOv11n deployed on a Raspberry Pi 5 — a resource-constrained, low-cost single-board computer. The system proves that state-of-the-art deep learning need not be confined to GPU servers: with a well-curated dataset, the right lightweight architecture, and edge-first deployment thinking, powerful computer vision is accessible at minimal cost.

A custom 8,242-image, 13-species dataset was curated, annotated, augmented, and used to train a YOLOv11n model achieving mAP@50 of approximately 0.90 after 10 training epochs. The model runs entirely on Raspberry Pi 5 — capturing frames via Pi Camera, performing on-device inference, and displaying annotated detections in real time on an HDMI monitor — with no internet connection required.

Key insights gained: dataset diversity and quality are the most critical determinant of model generalization; the nano YOLO architecture strikes an excellent accuracy-efficiency balance on embedded ARM hardware; and the deployment of modern object detectors on Raspberry Pi 5 is not merely theoretical but practically viable and accessible.

10. Future Scope

6. **Dataset Expansion:** Increase species coverage beyond 13 and add images from real underwater/aquarium environments with variable lighting, partial occlusion, and motion blur to improve generalization.
7. **Real-Time Video Tracking:** Extend from still-image detection to continuous video stream processing with multi-object tracking (MOT) — tracking individual fish across frames, counting populations, and analyzing behavior patterns.
8. **Model Optimization:** Apply INT8 quantization, weight pruning, and NCNN format conversion (up to 62% latency reduction on ARM64) to achieve 13+ FPS video-rate detection on Raspberry Pi 5 CPU.
9. **Hardware Acceleration:** Integrate NPU/TPU accelerators such as Google Coral USB or the Raspberry Pi AI Camera (IMX500) for substantial throughput improvements while maintaining the low-cost, edge-native model.
10. **Cloud-Edge Integration:** Hybrid architecture where Raspberry Pi handles local real-time detection while aggregated detection logs, analytics dashboards, and model updates are managed remotely via cloud services.
11. **Advanced Aquaculture Features:** Fish counting and density estimation, health anomaly detection (disease signs, abnormal swimming behavior), and automated alerts for farm operators when abnormal activity is detected.
12. **Multi-Camera Deployment:** Scale the system to support multiple camera nodes across a fish farm or research facility, each running independent on-device inference with centralized result aggregation.

11. References

- [1] Aung, T. et al. (2024). Artificial intelligence methods used in aquaculture: A systematic literature review. *Journal of the World Aquaculture Society*. DOI: 10.1111/jwas.13107
- [2] AquaYOLO (2025). Advanced YOLO-based fish detection for real-time monitoring in South Indian aquaculture ponds. *Nature Scientific Reports*. <https://www.nature.com/articles/s41598-025-89611-y>

- [3] Multi-class framework for fish species classification using YOLO (2024). PMC / NCBI. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12900365/>
- [4] YOLOv8-TF: Transformer-Enhanced YOLOv8 for Underwater Fish Species Recognition (2025). PMC. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11946109/>
- [5] Adhikary, S. et al. (2025). Fish species identification on low resolution using ESRGAN, YOLO and VGG-16. PeerJ Computer Science. DOI: 10.7717/peerj-cs.2860
- [6] Review of deep learning in aquatic animal husbandry (2025). PMC. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12453753/>
- [7] Simsek, F. (2026). Real-Time Object Detection on Raspberry Pi 5: YOLOv11 vs YOLOv8 Under CPU-Only Constraints. DOI: 10.21203/rs.3.rs-8584571/v1
- [8] Almstedt, A. et al. (2025). AntPi: A Raspberry Pi edge-cloud system for real-time species detection using YOLO. ScienceDirect — Ecological Informatics.
- [9] Benchmarking Deep Learning Models for Object Detection on Edge Computing Devices (2024). arXiv:2409.16808.
- [10] Energy efficiency of YOLOv8 and RT-DETR on edge devices (2026). Nature Scientific Reports. <https://www.nature.com/articles/s41598-026-46453-6>
- [11] Raspberry Pi Foundation (2024). Deploying Ultralytics YOLO models on Raspberry Pi. <https://www.raspberrypi.com/news/deploying-ultralytics-yolo-models-on-raspberry-pi-devices/>
- [12] LearnOpenCV (2025). YOLO11 on Raspberry Pi: Optimizing Object Detection for Edge. <https://learnopencv.com/yolo11-on-raspberry-pi/>
- [13] Terven, J. et al. (2023). A Comprehensive Review of YOLO Architectures: From YOLOv1 to YOLOv8 and YOLO-NAS. Machine Learning and Knowledge Extraction, 5, 1–31.
- [14] Barui, S. et al. (2018). Underwater object detection using YOLOv3. International Conference on Advanced Computing.
- [15] Mathias, A. et al. (2022). Aquatic species detection using YOLOv3 on UWA, DeepFish, Fish4Knowledge datasets.
- [16] Ultralytics (2024). YOLOv11 Documentation. <https://docs.ultralytics.com>

[17] PyTorch Documentation (2024). <https://pytorch.org/docs>

[18] GitHub Repository: AyushSriv06/Machli_project.
https://github.com/AyushSriv06/Machli_project